```
import cpmpy as cp
import json

def find_folding(Opt:dict[set])->list[dict]:
  """ find all folding relations of the optiongraph Opt """
  V=sorted(list(Opt.keys()))
  k=len(V)
  mdl=cp.Model()

  # class number
  cl={p:cp.intvar(1,k) for p in V}

  # partition
  mdl+=cl[V[0]]==1
  for i in range(1,len(V)):
    mdl+= cl[V[i]] <= 1+cp.max([cl[V[j]] for j in range(i)])

  # arrow in the quotient rulegraph
  def Qar(A,B):
    return cp.all([(cl[a]==A).implies
                   (cp.any([cl[b]==B for b in Opt[a]])) for a in V])

  # folding
  for a in V:
    for b in Opt[a]:
      mdl+=Qar(cl[a],cl[b]) | cp.any([cl[c]==cl[a] for c in Opt[b]])
      # mdl+=Qar(cl[a],cl[b]) | Qar(cl[b],cl[a])

  sols=[]

  def add_sol():
    result={p:cl[p].value() for p in V}
    sols.append(result)

  mdl.solveAll(display=add_sol)
  sols.sort(key=lambda d: (len(set(d.values())),
            json.dumps(d, sort_keys=True, separators=(',', ':'))))
  return sols

def print_sol(rel):
  """ print congruence relation as nontrivial classes """
  print('|',end='')
  kul=sorted(rel.keys())
  for a in set(rel.values()):
    vs=[v for v in kul if rel[v]==a]
    if len(vs)>1:
      print(*vs,end='|')
  print()
```

Figure 5.1

```
Opt={1: [2], 2: [3, 4, 5], 3: [6], 4: [5], 5: [6], 6: []}
rels=find_folding(Opt)
for rel in rels:
    print_sol(rel)
```

```
|1 4 6|3 5|
|1 4 6|
|3 5|4 6|
|3 5|
|4 6|
|
```

Figure 5.2

```
Opt={1: [2, 3], 2: [4, 5], 3: [4, 5], 4: [], 5: []}
rels=find_folding(Opt)
for rel in rels:
    print_sol(rel)
```

```
|1 4 5|2 3|
|1 4|2 3|
|1 5|2 3|
|2 3|4 5|
|1 4 5|
|2 3|
|1 4|
|1 5|
|4 5|
|
```

Figure 5.3

```
Opt={1: [3], 2: [4], 3: [5], 4: [5], 5: []}
rels=find_folding(Opt)
for rel in rels:
    print_sol(rel)
```

```
|1 2 5|3 4|
|1 2|3 4|
|1 2 5|
|1 5|3 4|
|2 5|3 4|
|3 4|
```

```
|1 5|
|2 5|
|
```

Figure 5.4

```
Opt={6: [5], 5: [4], 4: [3], 3: [2], 2: [1], 1: []}
rels=find_folding(Opt)
for rel in rels:
    print_sol(rel)
```

```
|1 3 5|2 4 6|
|1 3 5|2 4|
|1 3 5|2 6|
|1 3 5|4 6|
|1 3|2 4|
|1 3 5|
|1 3|
|
```

Figure 9.1

```
Opt={1: [3, 4], 2: [4], 3: [5, 6], 4: [6], 5: [], 6: []}
rels=find_folding(Opt)
for rel in rels:
    print_sol(rel)
```

```
|1 2 5 6|3 4|
|1 2|3 4|5 6|
|1 2 5 6|
|1 5 6|3 4|
|2 5 6|3 4|
|1 2 6|
|3 4|5 6|
|1 5 6|
|2 5 6|
|1 6|
|2 6|
|5 6|
|
```

Example 11.2: totative

```
import math
```

```
Opt={i:[j for j in range(1,i) if math.gcd(i,j)==1] for i in  range(1,10)}
print(Opt)
rels=find_folding(Opt)
print(len(rels))
for rel in rels:
    print_sol(rel)
```

```
{1: [], 2: [1], 3: [1, 2], 4: [1, 3], 5: [1, 2, 3, 4], 6: [1, 5], 7: [1, 2, 3,
4, 5, 6], 8: [1, 3, 5, 7], 9: [1, 2, 4, 5, 7, 8]}
20
|2 4 6 8|3 9|
|2 4 6 8|
|2 4 6|3 9|
|2 4 8|3 9|
|2 6 8|3 9|
|2 6|3 9|4 8|
|2 4 6|
|2 4 8|
|2 4|3 9|
|2 6 8|
|2 6|4 8|
|2 6|3 9|
|2 8|3 9|
|3 9|4 8|
|2 4|
|2 6|
|2 8|
|4 8|
|3 9|
|
```